

LLD Case Study 5: Multi-Floor Parking Lot System

Target Patterns: Singleton Pattern, Factory Pattern, Strategy Pattern

Key Engineering Focus: Multi-level spatial modeling, spot allocation algorithms, dynamic fee calculation, and thread-safe entry/exit gates.

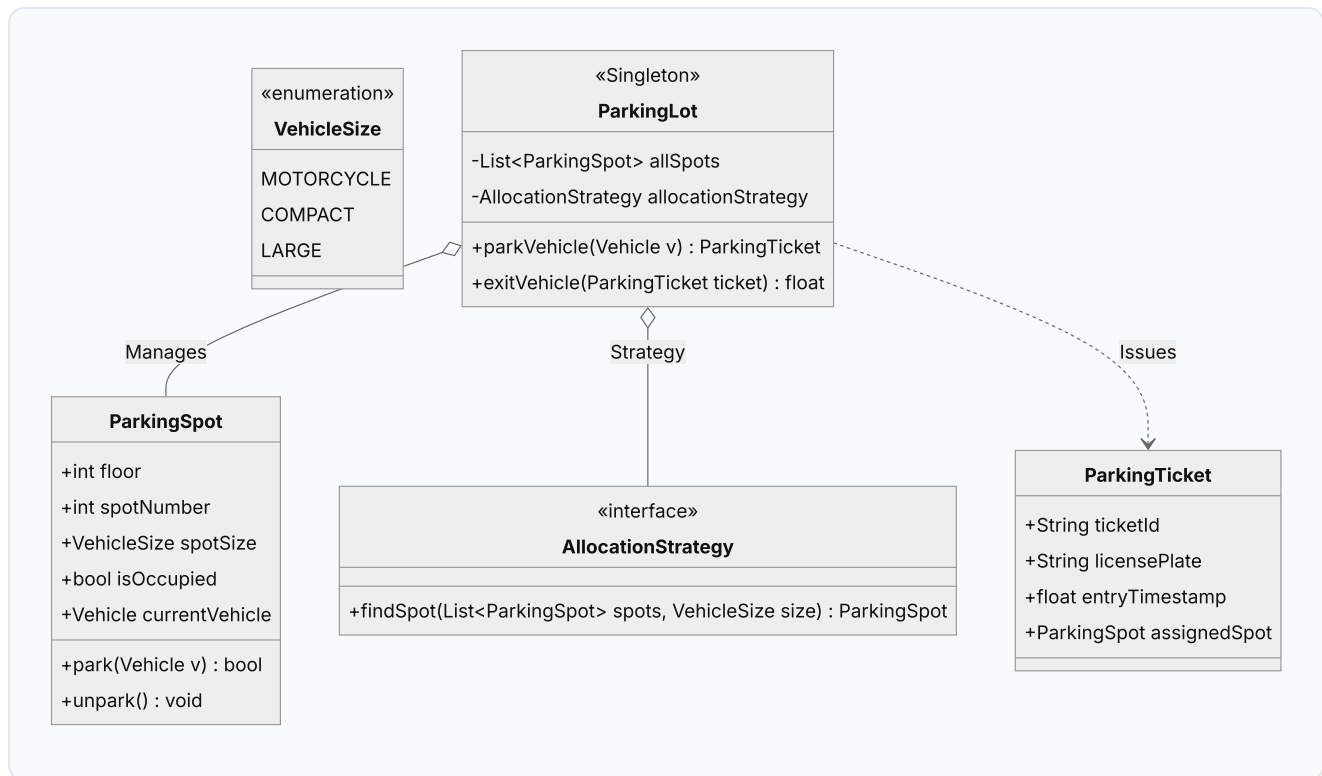
1. Problem Statement & Functional Requirements

Design an automated multi-floor commercial parking lot management system.

Requirements:

1. **Singleton Architecture:** The physical parking lot configuration is a system-wide Singleton.
 2. **Vehicle & Spot Hierarchies (Factory Pattern):** Supports Motorcycles, Cars, and Heavy Trucks fitting into Motorcycle, Compact, and Large spots respectively.
 3. **Spot Allocation Policy (Strategy Pattern):** Pluggable algorithms for spot assignment (e.g. Nearest to Entrance, Lowest Floor First).
 4. **Automated Ticketing & Billing (Strategy Pattern):** Issue timestamped tickets at entry gates and compute fees at exit based on duration and vehicle size multipliers.
-

2. Architecture & UML Class Diagram



3. Production-Grade Python Implementation

4. Edge Cases, Tests and Extensions

A bug the audit found, and the design issues behind it

The first version of this chapter set `LowestFloorFirstStrategy()` as the default strategy, a class that was never defined, so the demo crashed with `NameError` on the first car. The fix is to use the `NaiveLinearSearchStrategy` defined above. It is a good reminder that a case study is not finished until its driver has been run: the repository's audit script now flags code blocks that fail on a name defined nowhere in their file.

Concern	Status	Detail
Smallest fitting spot first	Good	The naive strategy sorts by floor, then spot size, then id, so a compact car takes a compact spot before a large one
Heap strategy loses spots	Bug	<code>find_spot</code> pops a spot from the heap; if the later <code>spot.park()</code> fails, nothing pushes it back, so a free spot disappears
Stale choice under concurrency	Weak	The strategy picks a spot without holding its lock; if another thread parks there first, <code>park_vehicle</code> returns <code>None</code> even though other spots are free
Fee uses the spot size	Questionable	A motorcycle in a large spot pays the large rate; most lots charge by vehicle type
<code>return_spot</code> never called on exit	Bug (heap only)	<code>exit_vehicle</code> frees the spot but does not return it to the heap strategy
Singleton	Works, hurts tests	Global state means tests must reset <code>ParkingLot._instance</code>

Tests

This block extends the implementation above (it uses `unittest.mock` to control the clock).

Extensions interviewers ask for

- Pricing by vehicle type and duration tiers:** pass the vehicle size, not the spot size, to a `PricingStrategy`; add tiers (first hour flat, then per 30 minutes, daily cap).
- Multiple entrances and exits:** the lot owns the allocation lock; each gate is a thin client holding a ticket printer, so gates never share state beyond the lot.
- EV charging and reserved spots:** add spot attributes (`has_charger` , `reserved_for`) and make eligibility a predicate list instead of the single `can_fit` rule.
- Reservations:** a `Reservation` holds a spot for a time window; the allocator excludes held spots, and a sweeper releases no-shows.
- Display board:** an observer updates free-spot counts per floor on every park and exit, using counters instead of scanning.

Follow-up questions

- Why is the naive search acceptable for a few hundred spots?* $O(N)$ over a few hundred items is microseconds; a heap per size becomes worthwhile at tens of thousands of spots, and only if

you fix the lost-spot bug above.

- *Is the Singleton needed?* A lot is naturally one object per site, but passing it in explicitly gives the same effect without global state and makes the tests simpler.
- *How do you make allocation race-free?* Allocate and park under one lock (or atomic compare-and-set on the spot) so the choice cannot go stale, rather than retrying afterwards.